

White Box Testing

Problems	Output	Debug Console	Terminal	Ports	
File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	45.1	13.88	40.54	45.36	
backend	91.48	0	66.66	91.48	
app.js	94.28	0	50	94.28	60-61
database.js	83.33	100	100	83.33	19-20
backend/config	100	100	100	100	
cloudinary.js	100	100	100	100	
backend/controllers	29.18	13.43	40	29.42	
adminController.js	49.35	50	53.84	49.35	...29,135-139,145-152
claimController.js	9.09	0	0	9.09	...32,141-209,218-230
foundItemPost.js	8.92	0	0	8.92	7-130
lostItemPost.js	8.47	0	0	8.47	7-143
searchItems.js	53.84	38.46	100	58.33	16-17,32-39,53,63-92
userSignUp.js	51.61	42.85	75	51.61	...82,108-109,115-161
backend/middleware	52	25	50	52	
auth.js	45.45	50	100	45.45	8-14
upload.js	57.14	0	0	57.14	19-27
backend/models	100	100	100	100	
claimModel.js	100	100	100	100	
foundItemModel.js	100	100	100	100	
lostItemModel.js	100	100	100	100	
pendingUserModel.js	100	100	100	100	
userRegModel.js	100	100	100	100	
backend/routes	85.18	100	0	85.18	
adminRoutes.js	100	100	100	100	
claimRoutes.js	100	100	100	100	
lfmsRouters.js	50	100	0	50	9-12,17-20
searchRoutes.js	100	100	100	100	
userRoutes.js	100	100	100	100	

White Box Test Report

Started: 2025-04-29 12:17:29

Suites (1)	Tests (20)
1 passed	20 passed
0 failed	0 failed
0 pending	0 pending

^ C:\Users\wajah\Desktop\Lost-and-Found-System\backend__test__\app.test.js				20.496s
User Registration & Authentication	should return 400 if email or password is missing	passed		0.49s
User Registration & Authentication	should return 200 for a new registration	passed		10.538s
User Registration & Authentication	should return 401 for invalid email on sign in	passed		0.381s
Lost & Found Item Posting	should return 401 or 404 if no token is provided when posting lost item	passed		0.091s
Lost & Found Item Posting	should return 401 or 404 if no token is provided when posting found item	passed		0.05s
Search Endpoints	should return 200 and results for search	passed		0.284s
Search Endpoints	should return 400 if lat/long are missing in nearby search	passed		0.15s
Admin Endpoints	should return 401 for invalid admin credentials	passed		0.144s
Admin Endpoints	should return analytics data for admin	passed		1.051s
Admin Endpoints	should return all users for admin	passed		0.251s
Admin Endpoints	should return all items for admin	passed		0.484s
Admin Endpoints	should return all claims for admin	passed		0.553s
Claim Controller	should return 400 or 401 if required fields are missing when creating a claim	passed		0.171s
Claim Controller	should return 400 or 401 for invalid itemId format	passed		0.046s
Claim Controller	should return 401 or 403 if not authenticated when getting finder claims	passed		0.167s
Claim Controller	should return 400, 401, or 403 for invalid claim status	passed		0.059s
Claim Controller	should return 401 or 403 if not authenticated when getting my claims	passed		0.113s
Lost Item Endpoints	should return all lost items	passed		0.133s
Lost Item Endpoints	should return 404, 401, or 403 for a non-existent lost item	passed		0.142s
Lost Item Endpoints	should return 401, 403, or 404 when deleting a lost item without auth	passed		0.151s

✓ What is Covered Well:

1. Functional Endpoints Testing:

- All major endpoints are covered with positive (e.g., `should return 200`) and negative (e.g., `should return 401 or 404`) test cases.
- User registration, authentication, lost/found item posting, claim handling, and admin functionality are thoroughly validated.
- All 20 test cases passed successfully, showing good endpoint behavior validation.

2. Code Coverage in Key Areas:

- Core files like `app.js` and `database.js` have high statement and line coverage (~83–94%).
 - Models and route files are 100% covered, which is excellent as they're critical to application logic and routing.
 - The `config` folder (e.g., `cloudinary.js`) is fully tested.
-

⚠ What is Not Covered and Why:

1. Controllers:

- Controllers such as `claimController.js`, `lostItemPost.js`, `foundItemPost.js`, etc., have **very low coverage (under 10%)**.
- Reason: These may contain internal logic or edge cases not triggered by endpoint tests. Tests likely focus on high-level flows without directly invoking or mocking these controllers.

2. Middleware:

- The `auth.js` and `upload.js` middlewares have low coverage (~45–57%).
- Reason: Some authentication flows and file upload conditions may not have been tested (e.g., malformed tokens or large/bad files).

3. **Branches:**

- Overall branch coverage is extremely low (13.88%), indicating that many conditional paths (`if`, `else`, `switch`) aren't being exercised.
- Reason: This could be due to lack of deep unit tests for edge cases or nested logic in controllers and middleware.

4. **Unreachable Code or Third-party Libraries:**

- Files like `cloudinary.js` or database-related logic might rely on external services, which are harder to unit test unless mocked/stubbed.
- Also, lines marked as uncovered may never run under typical test flows (e.g., logging, fallback handling).